DRIVEFLOW RENTALS

Database Design

CMPG 311

Group 14

Table of Contents

Group Information 3
SECTION 1: CONCEPTUAL DESIGN 4
1.1. Business Rules Derived from Operations 4
1.2. ER DIAGRAM 1
1.3. Entity Relationship (ER) Diagram 1
1.4. Normalisation Proof (3NF) 5
1.5. Advanced ER Features 7
SECTION 2: LOGICAL DATA MODEL 9
2.1. Logical Relations for All Entities 9
2.2. Normalisation — Evidence of 3NF Compliance 11
2.3. Referential Integrity Constraints 12
2.4. Logical Design 13

Group Information

Name & Surname	Student Number	Contact Information	Member Involvement
Kudzai Mudzingwa (Group Leader)	44177348	44177348@mynwu.ac.za +27 76 319 0045	Contributed
Xoliswa Sentse	37940066	37940066@mynwu.ac.za +27 81 459 5244	Contributed
Trevour Sithole	45485097	45485097@mynwu.ac.za +27 69 161 6943	Contributed
Edition Baloyi	43796540	43796540@mynwu.ac.za +27 79 300 2793	Contributed
Shekinah Khoza	44833563	44833563@mynwu.ac.za+ 27 68 252 3359	Contributed
Thato Mandelisa	34286799	34286799@mynwu.ac.za +27 69 110 2561	Contributed
Rifuwo Mbombi	40676099	40676099@mynwu.ac.za +27 66 539 1680	Contributed
Karabo Monyane	43391710	43391710@mynwu.ac.za +27 70 357 0728	Contributed

SECTION 1: CONCEPTUAL DESIGN

1.1. Business Rules Derived from Operations

The following business rules govern the operations of DriveFlow Rentals and have been directly derived from the system's operational procedures as described in the company situation analysis:

User & Authentication Rules

- A USER must have a unique email address and a valid driver's licence number before being registered in the system.
- A USER must have their password stored as a hashed value (PBKDF2) — plain-text passwords are never stored.
- Each USER is assigned exactly one ROLE (admin, staff, or customer). A USER cannot have more than one role simultaneously.
- A USER earns loyalty points through bookings, reviews, and on-time clean returns. Points may be redeemed as monetary discounts on future bookings.

Vehicle & Fleet Rules

- Each VEHICLE must have a unique VIN (Vehicle Identification Number) and a unique licence plate number.
- A VEHICLE belongs to exactly one category: Economy, SUV, or Luxury.
- A VEHICLE must always be in one of three statuses: Available, Rented, or Maintenance. A vehicle in Maintenance is not available for booking.
- A VEHICLE may be placed into zero or many MAINTENANCE_LOGs over its lifetime. Each maintenance log records the service type, cost, mileage, and next service date.

Booking Rules

- A CUSTOMER (USER with role = customer) may make zero or many BOOKINGS over time.

- A VEHICLE may appear in zero or many BOOKINGS, but at any given time, a vehicle that is 'Rented' may only be in one active CONFIRMED booking.
- Each BOOKING must reference exactly one USER and exactly one VEHICLE — it cannot exist without both.
- A BOOKING must have a pickup_date that is not in the past, and a return_date that is strictly after the pickup_date.
- The system prevents double-booking through a SQL overlap check: a new booking is rejected if the vehicle already has a confirmed booking where the date ranges overlap.
- A discount may optionally be applied to a BOOKING through a valid PROMO_CODE or by redeeming loyalty points. A discount cannot exceed the total booking amount.

Payment Rules

- Each BOOKING must progress to a PAYMENT before status changes from 'Awaiting Payment' to 'Confirmed'. A BOOKING may have at most one initial PAYMENT record.
- A PAYMENT must reference exactly one BOOKING. It records the payment method (Card, EFT, or PayFast), a unique transaction reference, and the amount paid.
- Additional charges (penalties) generate separate PENALTY records and are paid through the penalty payment system, not through the original BOOKING payment.

Penalty Rules

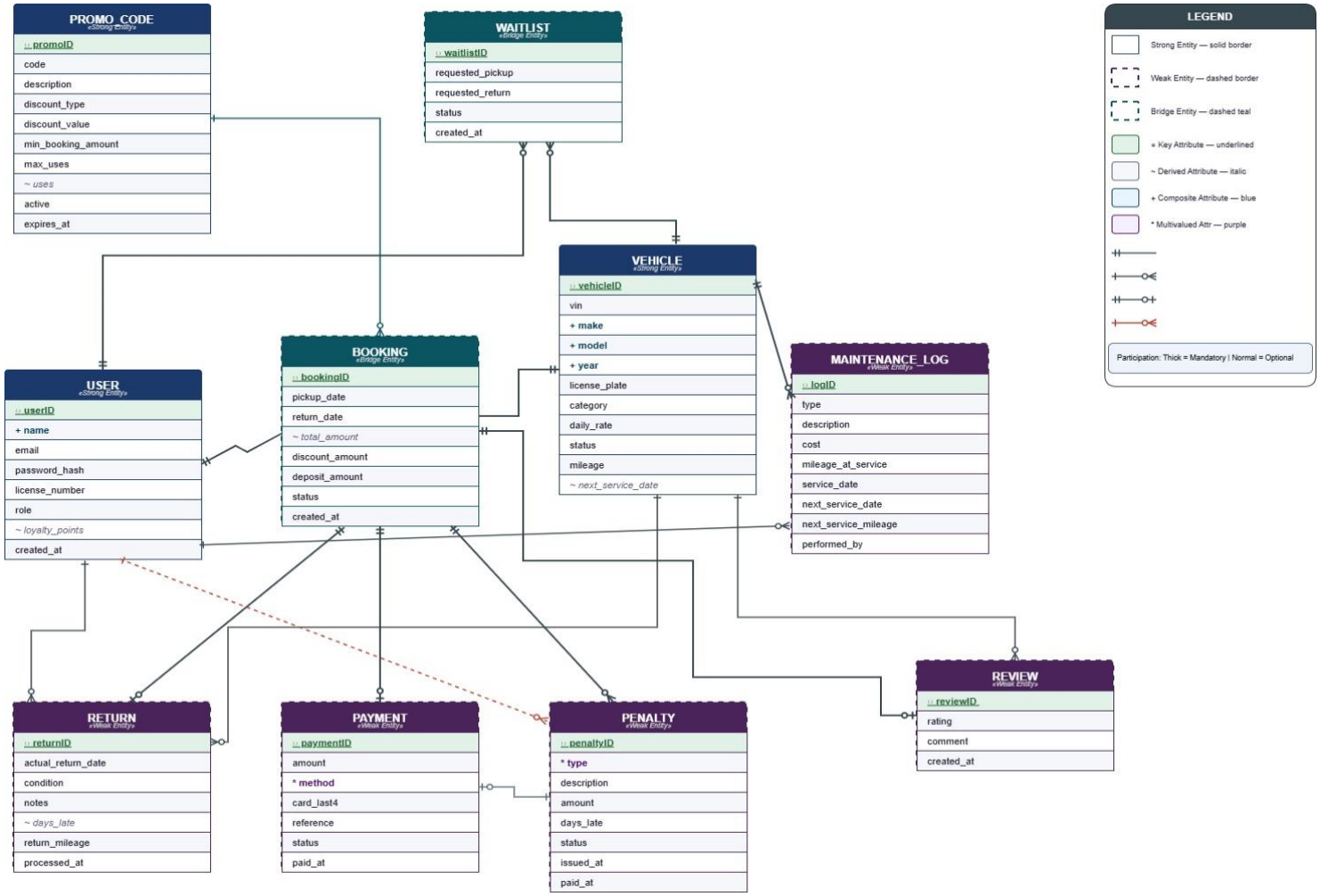
- A PENALTY is always linked to exactly one BOOKING and cannot exist without it (weak entity).
- A BOOKING may accumulate zero or many PENALTIES of different types: Late Return, Damage, Fuel, Cleaning, or Custom.
- Late return penalties are automatically calculated as: Days Late × Daily Rate × (1 + Surcharge%). Damage fees are flat-rate (Minor: R1,500; Major: R5,000).
- A PENALTY may be in one of three states: Unpaid, Paid, or Waived. An admin user may waive a penalty with a recorded reason.

Return, Review & Other Rules

- A RETURN record is created when an admin processes a vehicle return. It records the actual return date, vehicle condition, return mileage, and the number of days late.

- A REVIEW may only be submitted by the customer who holds the BOOKING, and only after the booking status is 'Returned'. Each booking may have at most one review.
- A PROMO_CODE has a maximum number of uses, an optional expiry date, a discount type (percent or flat), and an optional minimum booking amount.
- A USER may join a WAITLIST for a specific VEHICLE for desired dates if that vehicle is currently rented. The WAITLIST acts as a bridge entity between USER and VEHICLE.

1.2. ER DIAGRAM



1.3. Entity Relationship (ER) Diagram

The ER diagram for the DriveFlow Rentals system is described below. The diagram has been normalised to Third Normal Form (3NF) and depicts all required elements including entities, attributes, relationships, cardinality, participation, and advanced ER features.

1.3.1. Entities

The system consists of the following primary entities:

- *Strong Entities:* USER, VEHICLE, and PROMO_CODE.
- *Weak Entities:* PAYMENT, PENALTY, RETURN, MAINTENANCE_LOG, and REVIEW.
- *Bridge/Composite Entities:* BOOKING and WAITLIST.

1.3.2. Entities with Attributes, Primary Keys & Foreign Keys

USER

- Primary Key: userID
- Foreign Keys: None
- Regular & Derived Attributes: name, email, password_hash, license_number, role, loyalty_points (derived), created_at

VEHICLE

- Primary Key: vehicleID
- Foreign Keys: None
- Regular & Derived Attributes: vin, make, model, year, license_plate, category, daily_rate, status, mileage, next_service_date (derived)

BOOKING

- Primary Key: bookingID
- Foreign Keys: userID, vehicleID

- Regular & Derived Attributes: pickup_date, return_date, total_amount (derived), discount_amount, promo_code, deposit_amount, status, created_at

PAYMENT

- Primary Key: paymentID
- Foreign Keys: bookingID
- Regular & Derived Attributes: amount, method, card_last4, reference, status, paid_at

PENALTY

- Primary Key: penaltyID
- Foreign Keys: bookingID, issued_by
- Regular & Derived Attributes: type, description, amount, days_late, status, issued_at, paid_at, notes

RETURN

- Primary Key: returnID
- Foreign Keys: bookingID, processed_by
- Regular & Derived Attributes: actual_return_date, condition, notes, days_late, return_mileage, processed_at

REVIEW

- Primary Key: reviewID
- Foreign Keys: bookingID, userID, vehicleID
- Regular & Derived Attributes: rating, comment, created_at

PROMO_CODE

- Primary Key: promoID
- Foreign Keys: None
- Regular & Derived Attributes: code, description, discount_type, discount_value, min_booking_amount, max_uses, uses, active, expires_at, created_at

MAINTENANCE_LOG

- Primary Key: logID
- Foreign Keys: vehicleID
- Regular & Derived Attributes: logged_by, type, description, cost, mileage_at_service, service_date, next_service_date, next_service_mileage, performed_by, created_at

WAITLIST

- Primary Key: waitlistID
- Foreign Keys: userID, vehicleID
- Regular & Derived Attributes: requested_pickup, requested_return, status, created_at

1.3.3. Relationships, Cardinality & Participation

The connections between these entities are governed by specific business rules and cardinalities:

Core Booking Relationships

- USER to BOOKING: One USER can have zero or many bookings (0..*), but every BOOKING must be linked to exactly one USER (1..1). This is mandatory total participation for the booking.
- VEHICLE to BOOKING: A VEHICLE can appear in zero or many bookings (0..*), but a BOOKING must reference exactly one VEHICLE (1..1).
- PROMO_CODE to BOOKING: A PROMO_CODE can be applied to many bookings, while a BOOKING optionally uses one promo code.

Weak Entity Dependencies

- BOOKING to PAYMENT, PENALTY, RETURN, REVIEW: These entities are weak because they cannot exist without a parent BOOKING.
    - A booking has at most one initial payment.
    - A booking can have zero or many penalties.
    - A booking has at most one review, submitted only after the status is 'Returned'.

Maintenance & Administrative Relationships

- VEHICLE to MAINTENANCE_LOG: A VEHICLE may have many maintenance logs over time, but each log belongs to one specific vehicle.
- USER (Admin) to PENALTY: This is a recursive relationship where a USER with an admin role issues a penalty to another user (customer).

Referential Integrity Summary

To maintain data consistency, the following rules apply:

- Restrict Delete: You cannot delete a USER or VEHICLE if they are linked to active BOOKINGS.
- Cascade Delete: If a BOOKING is deleted, its associated PAYMENT, PENALTY, RETURN, and REVIEW records are automatically removed.

1.4. Normalisation Proof (3NF)

The DriveFlow Rentals schema has been designed and verified to satisfy Third Normal Form (3NF):

First Normal Form (1NF)

- All attribute values are atomic — no repeating groups or multi-valued cells in any relation.
- The penalty type is stored as a single VARCHAR value per row, not as a set in one cell.
- Each table has a defined primary key that uniquely identifies every row.

Second Normal Form (2NF)

- All non-key attributes in every table are fully functionally dependent on the entire primary key.
- BOOKING: total_amount depends on both the userID (for discount) and vehicleID (for daily_rate) via the relationship — it is fully dependent on bookingID, the surrogate PK.
- PENALTY: all attributes (type, amount, days_late) depend on penaltyID, not just bookingID alone.
- REVIEW: rating and comment depend on reviewID, not on bookingID or vehicleID independently.

Third Normal Form (3NF)

- No transitive dependencies exist. Each non-key attribute depends only on the primary key, not on another non-key attribute.
- Vehicle daily_rate is stored in VEHICLE, not duplicated in BOOKING. BOOKING only stores the computed total_amount.
- User name and email are stored in USER only. BOOKING references userID — name is never stored redundantly in BOOKING.
- Penalty types and their descriptions are stored per-row in PENALTY; there is no separate type lookup table because the type domain is an enumerated set with no additional attributes.

- PROMO_CODE details (discount value, type) are stored only in PROMO_CODE. BOOKING stores only the code string as a reference, not the discount details redundantly.

1.5. Advanced ER Features

	Feature	Where in Schema	Explanation
1	**Optional / Mandatory Relationships**	`USER → BOOKING; VEHICLE → BOOKING`	A USER optionally has bookings (0..*), but each BOOKING MUST have a USER (mandatory total participation). Similarly, a VEHICLE optionally appears in bookings, but each BOOKING MUST reference exactly one VEHICLE.
2	**Weak Entities & Identifying Relationships**	`PAYMENT, PENALTY, RETURN, MAINTENANCE_LOG`	PAYMENT, PENALTY, and RETURN cannot exist without their parent BOOKING — they are weak entities identified by the bookingID foreign key. MAINTENANCE_LOG is a weak entity of VEHICLE. Their primary keys are partial keys that become unique only within the context of their owner.
3	**Recursive Relationship**	`USER → PENALTY (issued_by)`	The PENALTY entity has an issued_by attribute that references back to the USER entity (specifically an admin-role user). This creates a recursive relationship where a USER (admin) issues PENALTY records that are associated with other USERs (customers) through their bookings.
4	**Bridge / Composite Entities**	`BOOKING; WAITLIST`	BOOKING is a bridge/composite entity that resolves the many-to-many relationship between USER and VEHICLE. WAITLIST is a second bridge entity linking the same two strong entities for the waitlist feature, with additional attributes (requested_pickup, requested_return, status).
5	**Composite Attributes**	`VEHICLE.vehicle_description; USER.name`	The vehicle's descriptive identity is a composite attribute made up of three component attributes: make, model, and year. Together they form the full vehicle description (e.g., '2023 Toyota Fortuner'). USER.name conceptually represents a composite of first name and last name stored as a single field.
6	**Multivalued Attributes**	`PENALTY.type; PAYMENT.method`	PENALTY.type can hold values from a set of valid types: {Late Return, Damage, Fuel, Cleaning, Custom}. PAYMENT.method takes values from {Card, EFT, PayFast}. These represent multivalued

	Feature	Where in Schema	Explanation
			domains where each instance holds one value from a defined set of possible values.
7	**Derived Attributes**	`BOOKING.total_amount; VEHICLE.next_service_date; USER.loyalty_points`	BOOKING.total_amount is derived: (return_date − pickup_date) × daily_rate − discount_amount. VEHICLE.next_service_date is derived from the most recent MAINTENANCE_LOG entry. USER.loyalty_points is derived as the cumulative sum of all LOYALTY_TRANSACTION amounts for that user.

SECTION 2: LOGICAL DATA MODEL

2.1. Logical Relations for All Entities

The logical data model presents each entity as a formal relation. Notation conventions:

- **Primary Keys: Bold and underlined**
- **Foreign Keys: Bold and italic**
- Regular attributes: Normal weight

PK = Bold Underline	FK = Bold Italic	Regular = Normal weight

Relations:

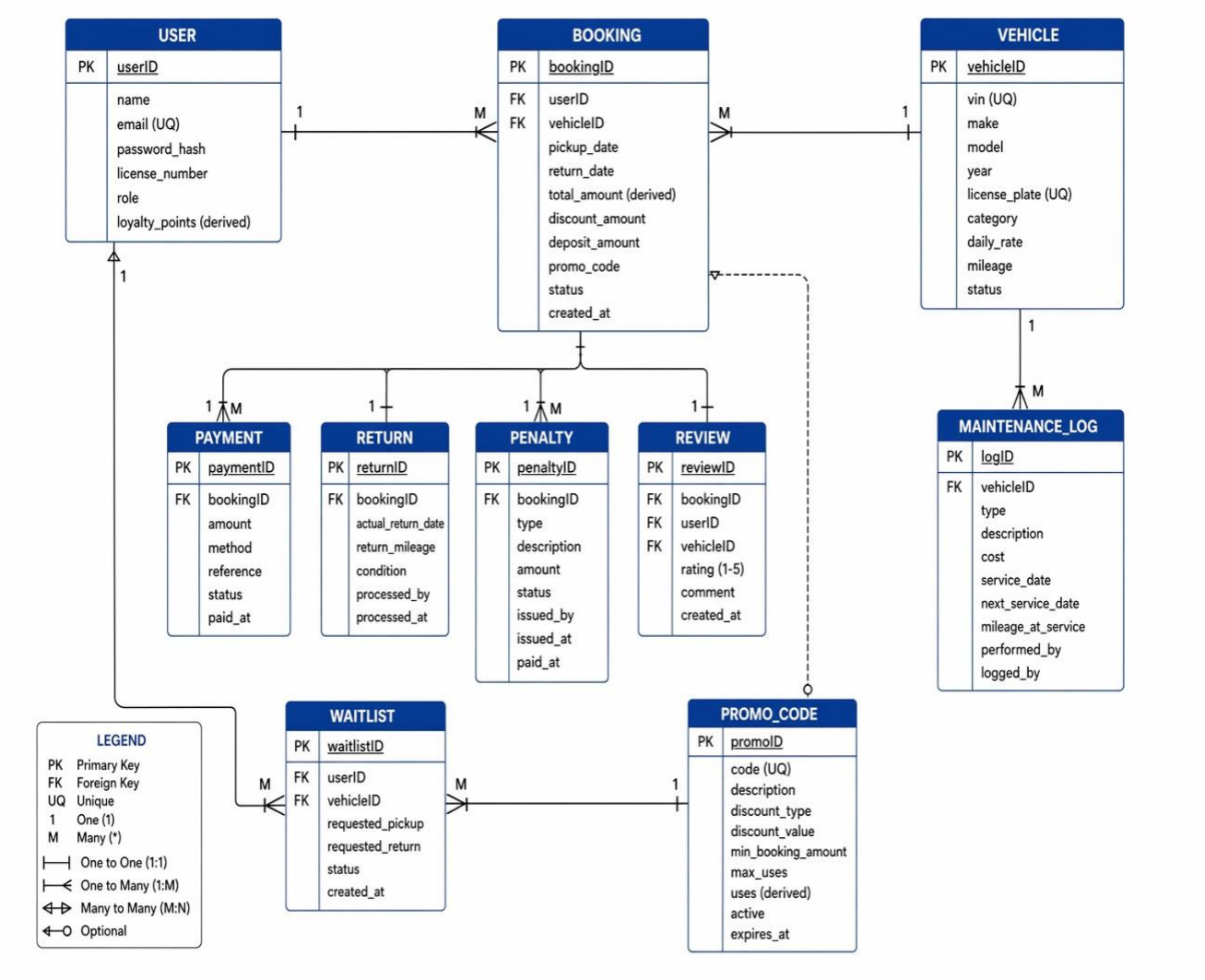
USER	**userID**, name, email, password_hash, license_number, role, loyalty_points, created_at
VEHICLE	**vehicleID**, vin, make, model, year, license_plate, category, daily_rate, status, mileage, next_service_date
BOOKING	**bookingID**, userID, vehicleID, pickup_date, return_date, total_amount, discount_amount, promo_code, deposit_amount, status, created_at
PAYMENT	**paymentID**, bookingID, amount, method, card_last4, reference, status, paid_at
PENALTY	**penaltyID**, bookingID, issued_by, type, description, amount, days_late, status, issued_at, paid_at, notes

RETURN	**returnID**, bookingID, processed_by, actual_return_date, condition, notes, days_late, return_mileage, processed_at
REVIEW	**reviewID**, bookingID, userID, vehicleID, rating, comment, created_at
PROMO_CODE	**promoID**, code, description, discount_type, discount_value, min_booking_amount, max_uses, uses, active, expires_at, created_at
MAINTENANCE_LOG	**logID**, vehicleID, logged_by, type, description, cost, mileage_at_service, service_date, next_service_date, next_service_mileage, performed_by, created_at
WAITLIST	**waitlistID**, userID, vehicleID, requested_pickup, requested_return, status, created_at

2.2. Normalisation — Evidence of 3NF Compliance

Normal Form	Requirement	Evidence in DriveFlow Schema
1NF	**No repeating groups; all values atomic; PK defined**	Each attribute holds a single atomic value. PENALTY.type holds one type per row — no comma-separated lists. Every relation has a single-column surrogate primary key (auto-increment INT).
2NF	**No partial dependencies on a composite PK**	All tables use surrogate integer PKs (not composite). Therefore partial dependency is structurally impossible. All non-key attributes are fully dependent on the single PK column.
3NF	**No transitive dependencies**	daily_rate is in VEHICLE, not BOOKING. customer name is in USER, not BOOKING. Discount details are in PROMO_CODE, not BOOKING. No non-key attribute depends on another non-key attribute — all depend directly and only on the PK.
BCNF	**Every determinant is a candidate key**	In REVIEW, the combination (bookingID, userID) is constrained to be unique, making bookingID alone a candidate key since each booking belongs to one user. No non-trivial functional dependency violates BCNF in any relation.

2.3. Referential Integrity Constraints

Foreign Key	References	On Delete	Note
BOOKING.userID	USER(userID)	RESTRICT	Cannot delete user with active bookings
BOOKING.vehicleID	VEHICLE(vehicleID)	RESTRICT	Cannot delete vehicle with bookings
PAYMENT.bookingID	BOOKING(bookingID)	CASCADE	Payment deleted with booking
PENALTY.bookingID	BOOKING(bookingID)	CASCADE	Penalties deleted with booking
PENALTY.issued_by	USER(userID)	SET NULL	Preserve penalty if admin deleted
RETURN.bookingID	BOOKING(bookingID)	CASCADE	Return record deleted with booking
RETURN.processed_by	USER(userID)	SET NULL	Preserve return if staff deleted
REVIEW.bookingID	BOOKING(bookingID)	CASCADE	Review deleted with booking
REVIEW.vehicleID	VEHICLE(vehicleID)	RESTRICT	Cannot delete reviewed vehicle
MAINTENANCE_LOG.vehicleID	VEHICLE(vehicleID)	CASCADE	Logs deleted with vehicle
WAITLIST.userID	USER(userID)	CASCADE	Waitlist cleared if user deleted
WAITLIST.vehicleID	VEHICLE(vehicleID)	CASCADE	Waitlist cleared if vehicle deleted

2.4. Logical Design